**Full Stack Assignment – 50**

**Assignment ID:** FSD-50

**Title:** Data Retention and Archival Policy Management System

**Duration:** 3 Hours

## 1. Objective

Design and develop a **Data Retention and Archival Policy Management System** that manages records based on retention rules and automatically archives expired data.

The system must strictly enforce retention timelines and ensure **data persistence using a database**.

## 2. User Roles

**USER**

- Register and log in to the system
- Create records
- View own active and archived records (read-only for archived)

**ADMIN**

- Register and log in to the system
- Define and manage data retention policies
- View archived records and retention reports

## 3. Functional Requirements

**Authentication & Authorization**

- Implement user registration and login
- Use JWT-based authentication
- Protect all backend APIs using JWT
- Enforce role-based access control on APIs and frontend views

**Record Management**

- USER can create records with creation timestamp
- Records must be stored in the database
- Records must have a status: ACTIVE or ARCHIVED

**Retention Policy Management**

- ADMIN can define retention policies, such as:
  - Retention duration (e.g., 7 days, 30 days)
  - Record type or category
- Retention policies must be stored in the database

**Archival Processing**

- System must automatically mark records as ARCHIVED when retention duration expires
- Archived records must remain stored in the database
- Archived records must be read-only and not editable

**Retention Reporting**

- ADMIN can view:
  - Active records
  - Archived records
  - Records nearing retention expiry

**4. Business Rule (Critical)**

Once a record is archived due to retention expiry, it must not be editable or restored to active state by any user.

**5. Technical Constraints (Mandatory)**

- All record, policy, and archival data must persist in a database
- In-memory or hardcoded retention logic is not allowed
- Backend APIs must reflect actual database state
- Retention checks must use server-side time
- JWT authentication and role validation are mandatory
- Use environment variables for sensitive configuration

**6. Frontend Requirements**

- Login and registration pages
- Role-based dashboards for USER and ADMIN
- Record creation and listing view for USER
- Clear separation of active and archived records
- Retention policy configuration and reporting views for ADMIN

UI design is not evaluated; correctness of retention enforcement is the priority.

**7. Submission Requirements (Strict)**

**GitHub Repository**

- Backend source code
- Frontend source code
- Proper folder structure
- Clear and meaningful commit history

**Live Deployment**

- Live frontend deployment link
- Live backend deployment link
- Backend must be connected to the database

**README.md (Mandatory)**

The README file must include:

- Project overview
- Tech stack used
- User roles and permissions
- API endpoints
- Database schema or collections
- Steps to run the project locally
- Live frontend and backend links

**8. Evaluation Note**

Evaluation focuses on **retention rule enforcement, archival immutability, database-backed time logic, role control, and end-to-end execution**.